

Tenbou: Riichi Mahjong Score Tracker

Dokumentacja koncepcyjna projektu

Olejniczak Jeremi Jan

19 czerwca 2026

Contents

1	Cel projektu	3
2	Funkcjonalności	3
2.1	Tworzenie gry	3
2.2	Rozgrywka na żywo	3
2.3	Zakończenie rundy	4
2.4	Silnik liczenia punktów	4
2.5	Tabela wyników	5
2.6	Historia gier	5
2.7	Ekran Yaku	5
2.8	Logowanie i synchronizacja danych	6
3	Architektura	7
3.1	Struktura folderów	7
3.2	Event Sourcing	7
4	Moduły (features)	8
5	Struktury danych	8
5.1	Game	8
5.2	Event	8
6	Stos technologiczny	10
7	Niuanse i decyzje projektowe do pamięci	11

1 Cel projektu

Tenbou to aplikacja mobilna we Flutterze, mająca na celu wspomaganie graczy Riichi Mahjong poprzez:

- śledzenie punktów oraz progresu rozgrywki w czasie rzeczywistym,
- obliczanie wypłat punktowych na podstawie Han (dużych punktów) i Fu (małych punktów),
- przechowywanie historii rozgrywek,
- edukacyjny ekran prezentujący układy Yaku.

Projekt budowany jest w architekturze clean architecture, z podziałem na warstwy danych, domeny i prezentacji, zgodnie z wytycznymi prowadzącego.

2 Funkcjonalności

2.1 Tworzenie gry

- Gra zawsze przeznaczona jest dla 4 graczy.
- Przy tworzeniu gry przypisuje się graczy do ról A/B/C/D (patrz sekcja 5.2 — uzasadnienie tego oznaczenia) oraz do wiatrów startowych.
- Konfigurowalne parametry gry:
 - Długość gry (gameLength): East (Tonpuusen, 4 rundy), South (Hanchan, 8 rund), West, North — pełna pula czterech wiatrów do wyboru, bez specjalnej opcji „unlimited” (zapobiega to niejednoznacznościom przy „podwójnym przejściu” tego samego wiatru).
 - Punkty startowe (startingPoints), np. 25 000 lub 30 000.
 - Koniec gry przy punktach ≤ 0 (endAtZeroPoints) — flaga włącz/wyłącz.
- Gra może zostać zakończona wcześniej, niezależnie od ustawionej długości.

2.2 Rozgrzywka na żywo

Ekran aktywnej gry pokazuje w czasie rzeczywistym:

- kto jest aktualnie rozdającym (dealerem),
- przypisanie wiatrów do graczy w danej rundzie,
- numer rundy w formacie wiatr + numer, z honbą dodawaną tylko gdy > 0 :
 - przykład bez honby: Wschód 2
 - przykład z honbą: Wschód 2-3 (runda 2, honba 3)
- liczbę riichi sticków aktualnie leżących na stole,
- liczbę honb.

Cały ten stan jest rekonstruowany przez silnik gry na podstawie historii eventów — nie jest przechowywany jako osobne pole w bazie danych (patrz sekcja 4 — Event Sourcing).

2.3 Zakończenie rundy

Po zakończeniu każdej rundy gracz zaznacza jeden z czterech typów zakończenia:

Typ	Opis
Ron	Wygrana okraszona przez wyłożenie kafelka przez przegranego. Maksymalnie 3 zwycięzców (double/triple Ron), zawsze 1 przegrany.
Tsumo	Wygrana na własnym dociągu. Może wygrać więcej niż jedna osoba — do 3 graczy — ze względu na Nagashi Mangan, który traktowany jest jako odmiana Tsumo.
Ryuukyoku	Remis (wyczerpanie kafelków). Zapisywana jest lista graczy w tenpai — może być od 0 do 4 graczy. Transfer punktów między tenpai/noten zależy od tego, kto był dealerm.
Chonbo	Kara za błąd proceduralny. Jest to zawsze izolowany event — nic innego się w tej rundzie nie liczy. Może dotyczyć więcej niż jednego gracza w tej samej rundzie.

Niezależnie od typu zakończenia, w evencie rundy zapisywane są dodatkowo:

- lista graczy, którzy zadeklarowali Riichi,
- (dla Ryuukyoku) lista graczy w tenpai.

Konsekwencje dla stanu gry

Na podstawie typu zakończenia silnik gry automatycznie:

- przesuwając rundę do następnej (zmienia wiatr/numer rundy) lub utrzymuje dealera (jeśli wygrał lub był w tenpai — w zależności od reguł),
- dolicza honby,
- przenosi riichi sticki do następnej rundy (o ile nie zostały zebrane przez zwycięzcę),
- przelicza punkty wszystkich graczy.

Chonbo — szczegóły: honby zostają niezmienione, riichi sticki z tej rundy wracają do puli (cofane są tylko te zadeklarowane w bieżącej rundzie).

2.4 Silnik liczenia punktów

- Wprowadzanie wartości Han i Fu odbywa się wyłącznie przez steppery (przyciski +/-), nie przez wolne pole tekstowe — eliminuje to ryzyko wpisania niepoprawnej wartości.
 - Fu: 20 → 25 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100 → 110.
 - Han: 1 → 2 → 3 → 4 → (5+: próg Mangana — stepper Fu jest wtedy blokowany/chowany, ponieważ Fu nie ma już znaczenia).
 - Stepper Fu jest zawsze w pełni dostępny (20–110) niezależnie od typu wygranej (Tsumo/Ron).
 - Domyślne wartości startowe: 1 Han, 30 Fu.
- Yakuman: osobny toggle, który chowa steppery Han/Fu i pokazuje nowy stepper z zakresem 1×–6× (możliwość wielokrotnego Yakumana, np. podwójny Yakuman).
- Wypłata punktowa liczona jest na podstawie tabeli lookup (tabela standardowych wartości punktowych Riichi Mahjong — patrz Załącznik A), nie wzorem matematycznym.

- Kiriage Mangan (zaokrąglanie 4 Han 70 Fu oraz 3 Han 60 Fu do wartości Mangana) obsługiwane automatycznie.
- Tsumo — rozbiecie wypłaty:
 - Dealer wygrywa: wszyscy trzej pozostali gracze płacą tyle samo.
 - Non-dealer wygrywa: dealer płaci połowę całkowitej wypłaty, pozostali dwaj płacą po jednej czwartej.
- Honba: dodaje +100 punktów od każdego płacącego przy Tsumo (czyli +300 całkowitego dla zwycięzcy), oraz +300 punktów łącznie przy Ron (od przegranego).
- Chonbo — wysokość kary:
 - Non-dealer popełnia Chonbo: płaci 4000 dealerowi, 2000 każdemu z dwóch pozostałych non-dealerów.
 - Dealer popełnia Chonbo: płaci 4000 każdemu z trzech pozostałych graczy.
- Decyzja projektowa: nie automatyzujemy wyliczania Han na podstawie zaznaczanych checkboxów konkretnych Yaku — zmiennych jest za dużo (dodatkowe Fu za układ honorów, Dora, czerwone trójki, lokalne reguły punktacji za małe układy), więc gracz sam wylicza swoje Han/Fu i wpisuje sumę przez steppery.

2.5 Tabela wyników

- Dostępna w każdej chwili, niezależnie czy gra jest w trakcie czy zakończona.
- Prezentowana runda po rundzie (historia punktów, nie tylko stan bieżący).

2.6 Historia gier

- Możliwość przeglądania zakończonych gier i ich pełnych tabel wyników.

2.7 Ekran Yaku

Osobna zakładka główna (obok zakładki Gry), w pełni pionowy layout (bez wymuszania orientacji poziomej — 14–18 miniaturowych kafelków mieści się wygodnie nawet w dwóch rzędach 7+7 lub 9+9).

Widok listy:

- Taby sortujące po liczbie Han: 1 Han, 2 Han, ..., aż do Yakumana i podwójnego Yakumana.
- Każdy wiersz: ikona z liczbą Han (kolorowy sześciokąt z cyfrą, gwiazdka dla Yakumana) → nazwa (duża romanizacja japońska, mała nazwa angielska poniżej) → po prawej miniaturowa wizualizacja 14(+) kafelków lub opis tekstowy (np. dla Nagashi Mangan).

Widok szczegółowy (po kliknięciu):

- Duża nazwa japońska, nazwa angielska, pełna wizualizacja kafelków układu, opis, dodatkowe warunki/zastrzeżenia (np. „tylko closed hand”, częste błędy).
- Prezentowanych jest kilka przykładowych wariantów układu (z configu), nie tylko jeden losowy — rozwiewa to wątpliwości co do wariacji układu (np. All Green może mieć różne kombinacje trójek).

Format danych Yaku (asset projektu)

Plik konfiguracyjny YAML/JSON jako asset bundlowany z aplikacją (nie zewnętrzny, nie aktualizowany bez release'u). Grafiki kafelków: również jako assets (SVG).

Klocki w przykładach grupowane są w setach, a silnik wnioskuje typ na podstawie liczby elementów w grupie:

tiles:

- [1s, 2s, 3s] # sekwencja/trójka (3 elementy)
- [1m, 1m, 1m, 1m] # kan (4 elementy)
- [haku, haku, haku] # trójka identycznych (3 elementy)
- [east, east, east] # trójka honorów (3 elementy)
- [1p, 1p] # para (2 elementy)

- Wildcard **back** oznacza odwrócony, nieistotny kafelek (np. dla Small/Big Three Dragons, gdzie tylko trzy konkretne sety mają znaczenie). Cała grupa może być skrócona do **back** jako pojedynczej wartości, oznaczając „dowolny kompletny set”:

tiles:

- back
- [hatsu, hatsu, hatsu]

- Interpreter wczytuje tyle struktury, ile jest podane explicite, i samodzielnie dopełnia resztę do standardowego układu (4 sety + 1 para), jeśli sumowanie się nie zgadza.
- Wszystkie sety domyślnie oznaczane jako otwarte.
- Kan (4 kafelki w grupie) — orientacja wizualna (obrócone kafelki na bokach) nie jest kodowana w configu; silnik renderujący sam decyduje o wizualnej reprezentacji na podstawie długości grupy.
- Kokushi Musou (Trzyście Sierot) — niestandardowy układ; uproszczenie projektowe: ostatni kafelek w liście jest traktowany jako duplikat jednego z poprzednich trzynastu, bez specjalnego oznaczenia.
- Liczba kafelków w układzie jest zmienna (14–18) ze względu na Kany:
 - 1 Kan → 15 kafelków
 - 2 Kany → 16 kafelków
 - 3 Kany → 17 kafelków
 - 4 Kany → 18 kafelków (Suukantsu)

2.8 Logowanie i synchronizacja danych

- Logowanie do konta Google jest opcjonalne — aplikacja działa w pełni offline, logowanie potrzebne jest tylko do backupu/odtworzenia danych.
- Synchronizacja realizowana jako manualny backup/restore (nie automatyczny sync w tle) z wykorzystaniem Google Drive API.
- Zakres OAuth: **drive.appdata** — dostęp tylko do skrytego folderu danych aplikacji, nie do plików użytkownika na Dysku. Zwiększa to bezpieczeństwo i ułatwia uzasadnienie zakresu uprawnień.

3 Architektura

Projekt zbudowany w duchu clean architecture, z wyraźnym podziałem każdego feature'a na trzy warstwy:

- **data** — źródła danych, dostęp do bazy, mapowanie modeli,
- **domain** — logika biznesowa, modele domenowe, use case'y,
- **presentation** — widoki, widgety, zarządzanie stanem UI.

3.1 Struktura folderów

```
lib/  
  core/                # baza danych, połączenia, konfiguracja, utilsy  
  features/  
    (nazwa_featura)/  
      data/  
      domain/  
      presentation/  
  router/              # definicje ścieżek (go_router)  
  main.dart
```

3.2 Event Sourcing

Zamiast przechowywać bieżący stan gry (punkty, dealer, wiatry) jako mutowalne pole w bazie danych, projekt przechowuje wyłącznie eventy zakończenia każdej rundy. Pełny stan gry w danym momencie jest rekonstruowany przez silnik gry poprzez odtworzenie wszystkich eventów od początku gry.

Konsekwencje tego podejścia:

- Historia gry jest naturalnym efektem przechowywania danych (nie wymaga dodatkowej struktury).
- Stan żywej gry (aktualny dealer, riichi sticki na stole, numer rundy) jest trzymany w pamięci aplikacji (np. w providerze Riverpod) i odtwarzany na nowo przy każdym uruchomieniu/dostępie — nie jest persystowany jako osobny rekord.
- Cała logika rotacji dealera, naliczania honb i transferu riichi sticków żyje wyłącznie w silniku (game_engine), nigdy jako pole zapisane w evencie.

4 Moduły (features)

Moduł	Zawiera prezentację?	Opis
yaku_list	Tak	Lista Yaku, widoki szczegółowe, taby sortowania po Han. Czysto prezentacyjny + dane statyczne z assetów.
game_session	Tak	Aktywna rozgrywka: tworzenie gry, ekran rundy na żywo, wprowadzanie wyniku rundy, tabela wyników live.
game_history	Tak	Przeglądanie zakończonych gier i ich tabel wyników.
game_engine	Nie	Czysta logika domenowa: rekonstrukcja stanu z eventów, silnik punktacji (Han/Fu/Yakuman → wypłata), rotacja dealera, zarządzanie riichi stickami i honbami. Współdzielony między game_session i game_history.
auth	Tak (ekran logowania)	Logowanie Google (opcjonalne), backup/restore bazy danych na Google Drive (drive.appdata).

Uzasadnienie wydzielenia game_engine: game_session (gra aktywna) i game_history (przegląd zakończonych gier) współdzielą bardzo dużo logiki domenowej — przede wszystkim rekonstrukcję stanu z eventów oraz silnik punktacji. W clean architecture feature bez warstwy prezentacji jest w pełni dopuszczalny — funkcjonuje jako biblioteka domenowa.

5 Struktury danych

5.1 Game

```
{
  "id": "<ObjectBox default ID>",
  "eastPlayer": "<nazwa gracza A>",
  "southPlayer": "<nazwa gracza B>",
  "westPlayer": "<nazwa gracza C>",
  "northPlayer": "<nazwa gracza D>",
  "startingPoints": 25000,
  "gameLength": "East | South | West | North",
  "endAtZeroPoints": true,
  "createdAt": "<timestamp>"
}
```

- gameLength określa długość gry (Tonpuusen/Hanchan/pełne West/pełne North) — nazwa świadomie odróżniona od pola wind w evencie, by uniknąć kolizji koncepcyjnej (wiatr startowy gry vs. wiatr danej rundy).

5.2 Event

```
{
  "gameId": "<referencja do Game>",
  "index": 0,
  "wind": "East | South | West | North",
  "round": 2,
```



```

"honba": 0,
"endType": "Ron | Tsumo | Ryuukyoku | Chonbo",
"winners": [
  { "player": "A", "han": 1, "fu": 30, "yakuman": null },
  { "player": "B", "han": null, "fu": null, "yakuman": 2 }
],
"loser": "C",
"tenpai": ["A", "B", "C", "D"],
"riichiDeclarers": ["A", "B"],
"chonbo": ["D"]
}

```

Uwagi do struktury:

- Oznaczanie graczy A/B/C/D (a nie po wiatrach) — wybrane świadomie, ponieważ wiatry startowe mogłyby się mylić z wiatrami rundy, które zmieniają się co rundę, podczas gdy rola gracza w grze jest stała.
- wind + round reprezentują wiatr rundy, nie są redundantne — round jako prosty licznik (1–4) jest łatwiejszy do inkrementacji niż parsowanie złożonego stringa typu „East 2”.
- honba wyświetlana użytkownikowi tylko gdy > 0 (np. Wschód 2 vs. Wschód 2–3).
- winners — lista, bo Ron może mieć do 3 zwycięzców, Tsumo do 3 (z Nagashi Mangan). Pola han/fu są null gdy gracz wygrał Yakumanem (i odwrotnie — yakuman jest null, gdy wygrana standardowa) — explicit null zamiast 0, by uniknąć niejasności.
- loser — pojedynczy string lub null (dla Tsumo/Ryuukyoku/Chonbo, gdzie nie ma jednego „wyłożonego”).
- tenpai — lista wypełniana tylko dla Ryuukyoku.
- chonbo — lista, bo więcej niż jeden gracz może zostać ukarany w tej samej rundzie; gdy wypełnione, żadne inne pole wyniku się nie liczy (Chonbo jest zawsze izolowanym eventem).
- Riichi sticki na stole nie są przechowywane jako pole — silnik (game_engine) wylicza je na bieżąco, przechodząc po historii eventów danej gry.
- Aktualny dealer w danej rundzie nie jest przechowywany jako pole — to także wynik rekonstrukcji przez silnik (dealer może utrzymać swoją pozycję przez kilka honb z rzędu, co wymaga znajomości całej poprzedzającej historii, nie tylko bieżącego eventu).
- Koniec gry (np. wykrzycie „All Last” w trybie Hanchan) jest logiką silnika, wyliczaną z $gameLength + wind + round$, bez dodatkowej flagi w evencie.

6 Stos technologiczny

Kategoria	Biblioteka	Wersja	Uwagi
Framework	Flutter	3.44.x	Stabilna linia na czerwiec 2026
State management	flutter_riverpod	^3.3.2	Wybrany nad BLoC (mniej boilerplate'u, dobry fit dla projektu solo) i nad GetX (odradzany dla nowych projektów w 2026 ze względu na ryzyko utrzymania)
Code generation (Riverpod)	riverpod_generator	^4.0.4	Adnotacja <code>@riverpod</code> generuje providery automatycznie, eliminując ręczny boilerplate (klasy Provider/StateNotifierProvider pisane ręcznie)
Adnotacje Riverpod	riverpod_annotation	^4.0.3	Lekki pakiet z samą adnotacją <code>@riverpod</code> , wymagany razem z <code>riverpod_generator</code>
Immutable models	freezed	^3.2.5	Generuje niemutowalne klasy danych (<code>copyWith</code> , <code>==/hashCode</code> , <code>unsealedClasses</code>) — używane dla modeli Game i Event
Adnotacje freezed	freezed_annotation	n/n zgodna	Lekki pakiet z adnotacją <code>@freezed</code> , dodawany do dependencies (<code>freezed</code> sam jest <code>dev_dependency</code>)
Code generation (silnik)	build_runner	n/n zgodna	Wspólny silnik generujący kod dla <code>freezed</code> i <code>riverpod_generator</code> (pliki <code>.freezed.dart</code> , <code>.g.dart</code>)
Routing	go_router	^16.2.0	Oficjalny router zespołu Flutter
Baza danych	objectbox	^5.3.2	NoSQL, ACID, offline-first; dwie kolekcje: <code>games</code> i <code>events</code> (relacja ToOne/ToMany z <code>events</code> do <code>games</code>)
Baza danych (cd.)	objectbox_flutter_libs	n/n zgodna	Natywne biblioteki runtime
Grafika wektorowa	flutter_svg	^2.3.0	Renderowanie kafelków Mahjong jako SVG (np. z Wikimedia Commons)
Logowanie Google	google_sign_in	^6.3.0	Świadomie starsza wersja 6.x — wersja 7.x ma zgłoszony otwarty problem z autoryzacją Google Drive API (błąd 403 mimo poprawnego logowania)
Google APIs	googleapis	^16.0.0	Moduł <code>drive/v3</code> używany do <code>backup/restore</code>
Most autoryzacyjny	zob. przypis*	najnowsza zgodna	Łączy <code>google_sign_in</code> z <code>googleapis</code>
HTTP	http	^1.0.0	Własny <code>AuthClient</code> przekazujący nagłówki autoryzacyjne do <code>DriveApi</code>

*Pakiet mostkujący: `extension_google_sign_in_as_googleapis_auth`. „n/n zgodna” = najnowsza wersja zgodna z resztą zależności.

Uwaga dotycząca wersji: wszystkie wersje są zgodne ze stanem na czerwiec 2026 i powinny być traktowane jako punkt wyjścia — przed startem implementacji warto zweryfikować aktualne wersje na `pub.dev`, ponieważ ekosystem Flutter/Dart aktualizuje się często.

Uwaga dotycząca code generation: `freezed` i `riverpod_generator` korzystają ze wspólnego silnika `build_runner` — po dodaniu lub zmianie adnotacji (`@freezed`, `@riverpod`) konieczne jest uruchomienie `dart run build_runner build` (lub `watch` podczas developmentu), by wygenerować

pliki `.freezed.dart` / `.g.dart`.

7 Niuanse i decyzje projektowe do pamięci

1. Nagashi Mangan traktowany jest jako odmiana wygranej Tsumo (nie jako specjalny rodzaj remisu) — wpływa to na strukturę Event (pojawia się w winners, nie w osobnym polu).
2. Ron wieloosobowy (double/triple Ron) jest dozwolony — do 3 zwycięzców, zawsze 1 przegrany.
3. Chonbo jest zawsze izolowanym eventem — żadne inne pole wyniku (Han/Fu/zwycięzca/przegrany) nie ma znaczenia w evencie typu Chonbo.
4. Han/Fu wprowadzane wyłącznie przez steppery — świadoma decyzja by uniknąć błędów walidacji przy wolnym wpisywaniu liczb (np. niedozwolone wartości Fu).
5. Zaznaczanie konkretnych Yaku checkboxami zostało odrzucone jako metoda liczenia Han — zbyt wiele zmiennych wpływających na wynik (Dora, czerwone trójki, dodatkowe Fu za układ honorów) by to zautomatyzować w sposób niezawodny.
6. `gameLength` vs. `wind` (w evencie) — świadomie różne nazwy pól, by uniknąć kolizji koncepcyjnej między „długością/trybem gry” i „wiatrem aktualnej rundy”.
7. `google_sign_in v6.x` zamiast najnowszej `v7.x` — z powodu znanego, otwartego problemu z autoryzacją Google Drive API w `v7`.
8. Assets Yaku i grafiki kafelków są w pełni bundlowane z aplikacją (nie ładowane zewnętrznie) — prostsze, w pełni offline, bez zależności od zewnętrznego hostingu treści.
9. Brak własnego backendu — synchronizacja oparta wyłącznie na Google Drive API (`drive.appdata`), co eliminuje potrzebę serwera, kosztów hostingu i osobnej warstwy API.
10. Modele domenowe (Game, Event, Winner) generowane przez `freezed` — zapewnia to niemutowalność danych eventów (zgodnie z filozofią Event Sourcing, raz zapisany event nie powinien się zmieniać) oraz darmowy `copyWith/equality` bez ręcznego boilerplate’u.
11. Providery Riverpod definiowane przez adnotację `@riverpod` (`riverpod_generator`) zamiast ręcznego tworzenia klas Provider — mniej boilerplate’u, lepsza type-safety, łatwiejsze refaktoryzacje przy rozrastającym się drzewie zależności providerów.

Załącznik A: Tabela punktacji Riichi Mahjong (źródło)

Tabela wartości punktowych (Dealer/Non-Dealer, Ron/Tsumo, w zależności od liczby Han i Fu) oraz tabela Limit Hands (Mangan, Haneman, Baiman, Sanbaiman, Yakuman) — zgodnie ze standardowymi regułami Riichi Mahjong, dostarczona przez prowadzącego/użytkownika jako materiał źródłowy do implementacji silnika punktacji. Zasady wyliczania Fu (bazowe 20 Fu + punkty za triplet/kan w wariantie open/closed, +10 Fu za closed Ron, +2 Fu za Tsumo, +2 Fu za yakuhai pair, +2 Fu za edge/pair/closed wait, zaokrąglenie do najbliższej dziesiątki) oraz wyjątki (Chiitoitsu = zawsze 25 Fu; Pinfu+Tsumo = zawsze 20 Fu; Pinfu+Ron = zawsze 30 Fu) powinny zostać zaimplementowane jako stała tabela lookup w module game_engine.

Riichi Mahjong Scoring System

Dealer					Non-Dealer			
4 han	3 han	2 han	1 han	Ron Tsumo ND/Dealer	1 han	2 han	3 han	4 han
-	-	-	-	-	-	-	-	-
7800 2600	3900 1300	2100 700	-	20 fu	-	1500 400/700	2700 700/1300	5200 1300/2600
9600	4800	2400	-	25 fu	-	1600	3200	6400
9600 3200	4800 1600	-	-	30 fu	-	-	3200 800/1600	6400 1600/3200
11600*	5800	2900	1500	40 fu	1000	2000	3900	7700*
11700*	6000	3000	1500	50 fu	1100	2000	4000	7900*
3900*	2000	1000	500	60 fu	300/500	500/1000	1000/2000	2000/3900*
Mangan 12000 4000	7700	3900	2000	70 fu	1300	2600	5200	
	7800	3900	2100	80 fu	1500	2700	5200	
	2600	1300	700	90 fu	1600	3200	6400	
	9600	4800	2400	100 fu	1600	3200	6400	
	9600	4800	2400	110 fu	1600	3200	6400	
	3200	1600	800		400/800	800/1600	1600/3200	
	11600*	5800	2900		2000	3900	7700*	
	11700*	6000	3000		2000	4000	7900*	
	3900*	2000	1000		500/1000	1000/2000	2000/3900*	
	6800	3400			2300	4500		
	6900	3600			2400	4700		
	2300	1200			600/1200	1200/2300		
Mangan 8000 2000/4000	7700	3900			2600	5200		
	7800	3900			2700	5200		
	2600	1300			700/1300	1300/2600		
	8700	4400			2900	5800		
	8700	4500			3100	5900		
	2900	1500			800/1500	1500/2900		
	9600	4800			3200	6400		
	9600	4800			3200	6400		
	3200	1600			800/1600	1600/3200		
	10600	5300			3600	7100		
	10800	5400			3600	7200		
	3600	1800			900/1800	1800/3600		

*Use mangan score when mangan rounding is in effect.

Limit Hands

Dealer	Ron Tsumo ND/Dealer	Non-dealer
12000	5 han	8000
12000 4000	Mangan	8000 2000/4000
18000	6-7 han	12000
18000 6000	Haneman	12000 3000/6000
24000	8-10 han	16000
24000 8000	Baiman	16000 8000/4000
36000	11-12 han	24000
36000 12000	Sanbaiman	24000 6000/12000
48000	>13 han	32000
48000 16000	Yakuman	32000 8000/16000

To calculate fu, start with 20 fu and then consider the triplets and quads in your hand.

		Simples	Terminals and Honors
Triplet	Open	+2 fu	+4 fu
	Closed	+4 fu	+8 fu
Quad	Open	+8 fu	+16 fu
	Closed	+16 fu	+32 fu

Add 10 fu if won through a closed ron, 2 fu if won through tsumo, 2 fu if a yakuhai pair is present, and 2 fu if the wait was an edge wait, pair wait, or closed wait. Then round up to the nearest ten.

Chiitoitsu hands are always 25 fu.
Pinfu hands won through tsumo are always 20 fu.
Pinfu hands won through ron are always 30 fu.